**Indian Institute of Information Technology,**
**Design & Manufacturing, Kancheepuram**

# Thelawala

## A Vendor Aggregation Platform

**Mid Semester Review**

DS3001 — Prototyping & Testing



| Roll Number | Name |
|---|---|
| CS23B1062 | Anurag Sharma |
| CS23I1064 | Parth Pandey |
| CS23I1026 | Grishmank Parate |
| ME23B1009 | Shashwat Kumar Chandan |

**Mentor:** Dr. Narendran G.


March 2026

# Contents

**1 Problem Statement** **2**

**2 Objectives** **2**

**3 Assumptions** **2**

**4 Concept Design** **3**
4.1 Research-Driven Focus Areas . . . . . 3
4.2 Core Concept . . . . . 3
4.3 Addressing Hardware Constraints — The AUJAR Device . . . . . 3
4.4 Solution Architecture . . . . . 4

**5 Bill of Materials** **5**

**6 Prototype** **5**
6.1 Technology Stack . . . . . 6
6.2 Application Screenshots . . . . . 7
6.3 Key Implementation Details . . . . . 8

**7 Outcome & Inferences** **8**
7.1 Outcomes . . . . . 8
7.2 Inferences . . . . . 8

**8 Future Works & Development** **9**

**9 References** **9**

# 1. Problem Statement

Street vendors — commonly known as *thelawalas* — form a significant part of India's informal economy. Despite serving millions of customers daily, their operations remain entirely invisible in the digital space. Buyers have no way of knowing:

- When or where a specific vendor will appear in their locality.
- What products a nearby vendor is currently selling.
- Whether a vendor they prefer is even operating on a given day.

This unpredictability leads to missed purchase opportunities for buyers and lost revenue for vendors. Furthermore, no structured communication channel exists between a buyer and a thelawala — a buyer cannot signal purchase intent, and a vendor cannot notify regular customers of their presence.

From a vendor's perspective, thelawalas lack any digital tool tailored to their trade. Existing platforms (food delivery apps, ride-hailing services) are designed for formal businesses and do not map to the mobile, informal, and low-capital nature of street vending. Additionally, many thelawalas do not own a smartphone, making any phone-dependent GPS solution impractical for continuous all-day use.

**In summary:** the street-vending ecosystem suffers from a lack of real-time location awareness, product discoverability, and bidirectional communication — problems that no existing platform addresses.

# 2. Objectives

The primary objective of this project is to design and build a platform that bridges the information gap between street vendors and local buyers. The specific objectives are:

1. **Real-time vendor tracking:** Enable buyers to see the live location of nearby thelawalas on a map.
2. **Product-level discovery:** Allow buyers to search for vendors selling a specific item (e.g., "panipuri") and see results filtered by proximity.
3. **Buyer–vendor communication:** Provide a structured request mechanism where a buyer can request an item from a vendor, relaying their location so the vendor can approach them.
4. **Hardware independence:** Design a dedicated low-power GPS device (AUJAR) that can broadcast a vendor's location without requiring them to own or operate a smartphone.
5. **Proximity notification:** Notify users when a subscribed vendor enters their vicinity, even when the app is not active, through automated calling services.

# 3. Assumptions

The following assumptions were made during the design and development of the system:

1. Buyers (users) have access to an Android smartphone with GPS and internet connectivity.

2. The server has a stable internet connection and is reachable from the mobile applications.
3. Vendors may or may not own a smartphone; the system accounts for both scenarios through the AUJAR hardware device.
4. In the current prototype phase, the vendor's mobile phone GPS substitutes for the AUJAR device's GPS module.
5. All parties operate within a geographically bounded area where cellular data or Wi-Fi connectivity is available.
6. Vendors are willing to broadcast their location in exchange for increased customer visibility and sales.

# 4. Concept Design

## 4.1 Research-Driven Focus Areas

The concept design was driven by structured analysis using three frameworks:

**SNAC Analysis** identified four critical pain points in the street-vending ecosystem: unpredictable vendor availability, absence of any communication channel between buyers and vendors, complete vendor invisibility in the digital space, and missed revenue due to buyers being unaware of nearby vendors.

**Discovery Matrix** mapped unmet needs against existing alternatives and confirmed that no real-time tracking solution exists for informal street vendors, product-level search is entirely absent, and bidirectional buyer–vendor communication is an unaddressed gap.

**Interpretive Structural Modelling (ISM)** established a hierarchy of driving and dependent factors. Vendor location awareness, real-time communication infrastructure, and low-barrier hardware access emerged as foundational drivers (high driving power, low dependence), while purchase conversion, customer satisfaction, and vendor income growth were identified as dependent outcomes.

The convergent conclusion: the solution must focus on **(1) real-time vendor location tracking**, **(2) product discovery**, and **(3) a structured buyer–vendor communication channel**.

## 4.2 Core Concept

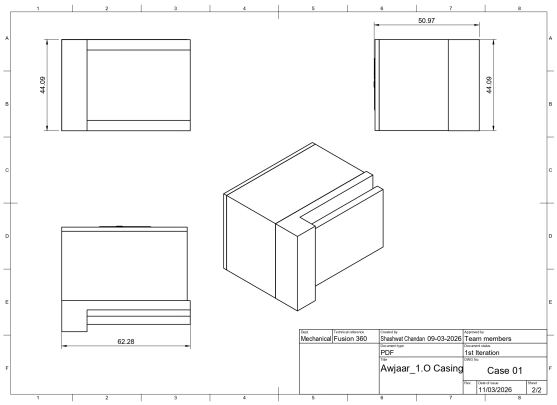
The platform provides four key capabilities:

1. **Track** — A user can see the live location of thelawalas on a map.
2. **Discover** — A user can view all vendors broadcasting near their current location.
3. **Search** — A user can search for vendors selling a specific product (by item name or vendor ID).
4. **Request** — A user can send a purchase request to a vendor; the buyer's location is relayed to the vendor, who can accept or reject the request.

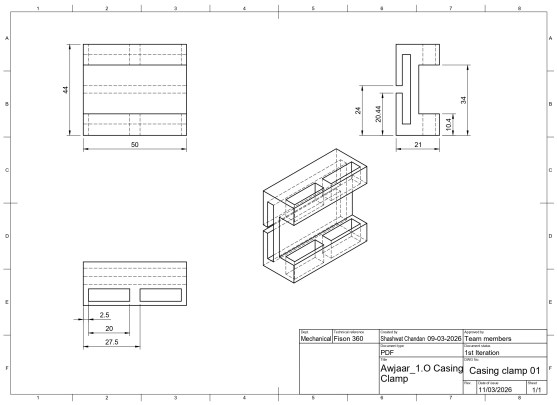
## 4.3 Addressing Hardware Constraints — The AUJAR Device

Continuous GPS tracking drains a smartphone's battery rapidly, and many thelawalas do not own a smartphone. To address both constraints, the concept introduces **AUJAR** — a purpose-built, low-power hardware device:

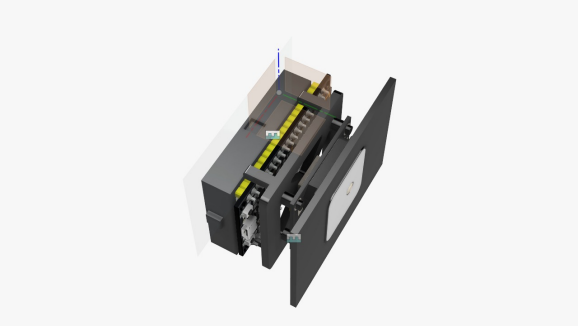
- Contains a GPS module that continuously acquires the vendor's position.
- Transmits coordinates over Bluetooth Low Energy (BLE) to any paired smartphone nearby.
- The paired phone acts as a bridge, forwarding GPS data to the Thelawala server.
- Requires no interaction from the vendor — simply turn on and mount on the thela.
- Designed to last a full work shift (8–12 hours) on a single charge.



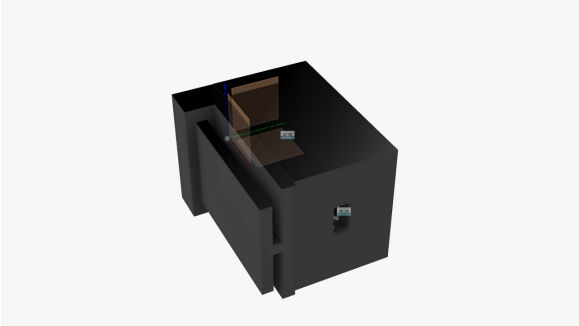

(a) AUJAR 2D casing design




(b) AUJAR 2D clamp design



(c) AUJAR 3D model



(d) AUJAR mount 3D model

Figure 1: AUJAR hardware device — 2D drawings and 3D models

## 4.4 Solution Architecture

The complete system is structured as a three-layer architecture with an optional hardware peripheral:

1. **Hardware Layer** — AUJAR device acquires GPS coordinates and transmits via BLE.
2. **Application Layer** — Two React Native mobile apps (buyer and vendor), communicating with the server via REST API and Socket.io WebSockets.
3. **Server Layer** — A Node.js/Express bridge server backed by MongoDB for persistence and Socket.io for real-time event relay.

The server uses JWT-based authentication for both REST and WebSocket channels. Vendor location data is persisted in MongoDB, and proximity filtering uses the Haversine formula to compute distances between GPS coordinates. The system maintains in-memory session maps and message queues so that requests and responses are not lost when either party is temporarily disconnected.

# 5. Bill of Materials

The Bill of Materials for the AUJAR hardware device is shown in Figure 2. It includes the GPS module, BLE-capable microcontroller, battery, enclosure, and mounting hardware.

| S.NO | Description of Component | Specification | Quantity | Unit price | Total Cost (INR) |
|---|---|---|---|---|---|
| 1 | ESP WROOM 32 MCU Module Version: 1.1 | ESP32 Based, WROOM Type Flash Memory (MB): 4MB Operating Voltage (VDC): 2.3 ~ 3.6 Operating Current (mA): 80 | 1.00 | ₹378.00 | ₹378.00 |
| 2 | NEO-6M GPS Module | NEO-6M GPS Module with EPROM – Normal Quality | 1.00 | ₹251.00 | ₹251.00 |
| 3 | 400 pin breadboard | Solderless 400 pin breadboard – Normal Quality – Without Packing | 1.00 | ₹36.00 | ₹36.00 |
| 4 | | | | | ₹0.00 |
| 5 | | | | | ₹0.00 |
| | | | | **Grand Total** | ₹665.00 |

Figure 2: Bill of Materials for the AUJAR device

# 6. Prototype

The working prototype consists of:

- A **Node.js bridge server** running Express for REST endpoints and Socket.io for real-time communication, connected to a MongoDB database.
- A **Vendor mobile application** (React Native / Expo) that allows vendors to register, manage inventory (add/edit/delete items), connect to GPS, broadcast their live location, and receive and respond to buyer requests.
- A **Buyer mobile application** (React Native / Expo) that allows buyers to register, view nearby broadcasting vendors on a dark-themed Leaflet map, search for vendors by product name or vendor ID, send item requests, and receive vendor responses with location pins.

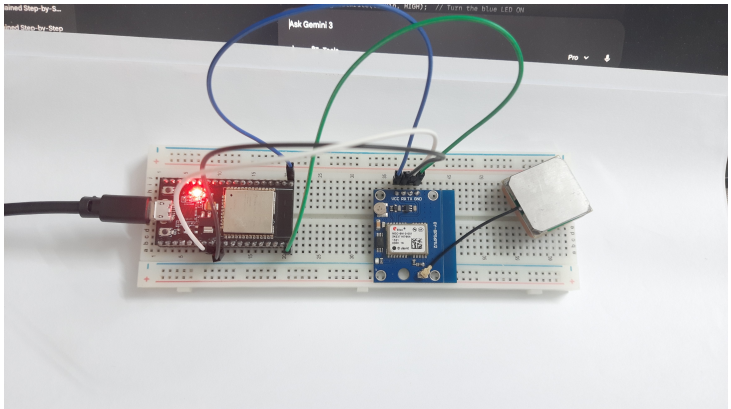


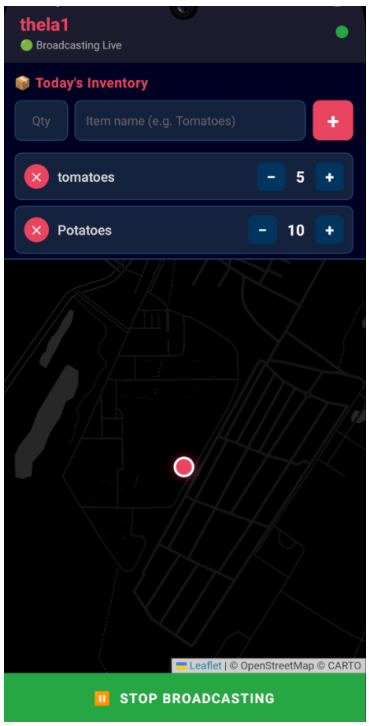
Figure 3: AUJAR hardware prototype

## 6.1 Technology Stack

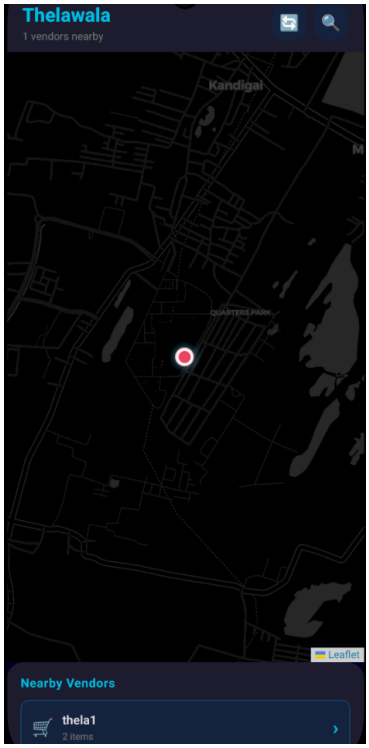
| Layer | Technology | Purpose |
|---|---|---|
| Server Runtime | Node.js | JavaScript runtime |
| API Framework | Express | REST endpoint routing |
| Real-time | Socket.io | WebSocket-based bidirectional communication |
| Database | MongoDB (Mongoose) | Persistent storage of vendors, clients, inventory |
| Authentication | JWT + bcrypt | Token-based auth (7-day expiry), password hashing |
| Mobile Framework | React Native (Expo) | Cross-platform mobile apps (TypeScript) |
| Maps | Leaflet.js (WebView) | Dark-themed interactive maps with custom markers |
| Location | Expo Location | Foreground GPS tracking on mobile |

Table 1: Technology stack

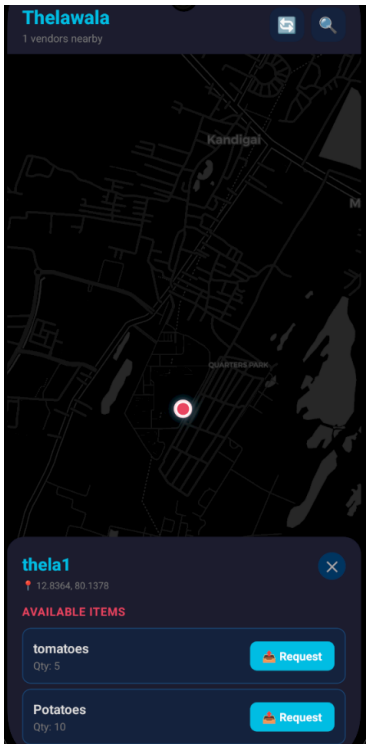
## 6.2 Application Screenshots



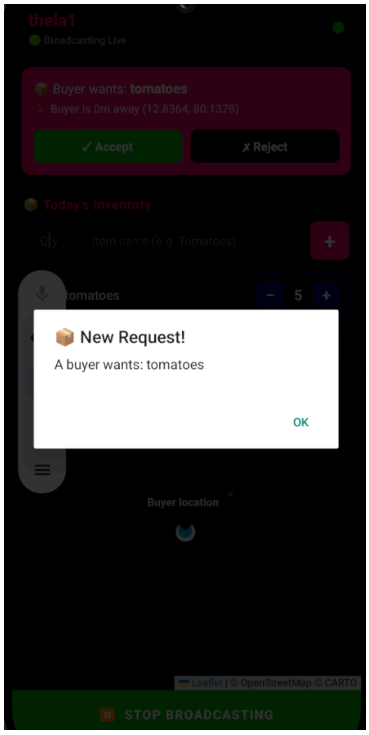

(a) Vendor home screen



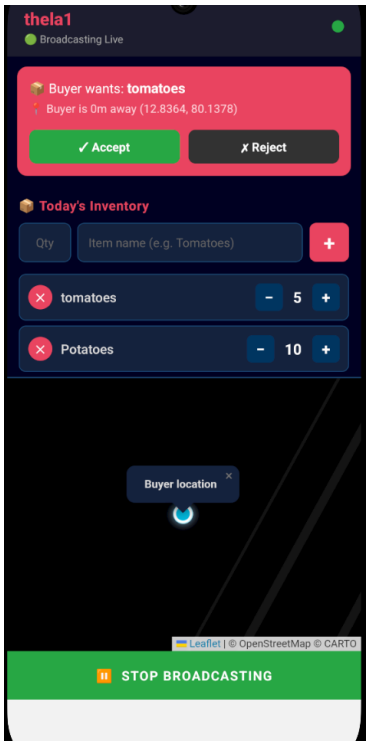

(b) User discovers vendors



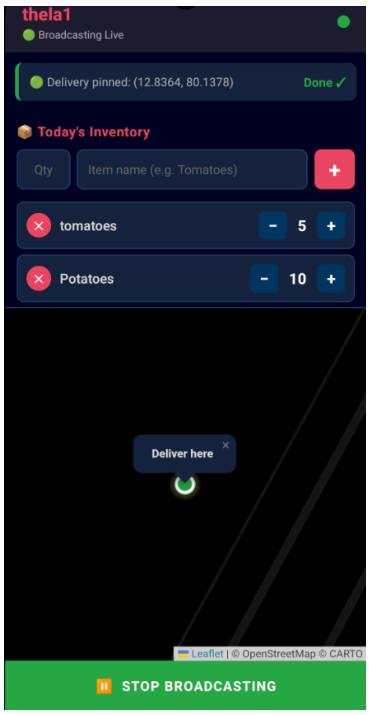

(c) User views vendor stock




(d) Vendor receives request




(e) Vendor sees buyer location




(f) User sees vendor pin

Figure 4: Application screenshots showing the complete request–response flow

### 6.3 Key Implementation Details

**Real-time Location Broadcast:** The vendor app acquires GPS coordinates every 5 seconds (1-metre distance threshold) and emits `vendor_location_update` events via Socket.io. The server persists the location and sets the vendor as "broadcasting."

**Nearby Vendor Discovery:** The buyer app polls the server every 15 seconds via a `get_nearby_vendors` socket event. The server queries all broadcasting vendors, filters them within a 5 km radius using the Haversine formula, and returns the list.

**Item Request Routing:** When a buyer sends a request, the server looks up the vendor's active socket session. If the vendor is online, the request is delivered immediately; if not, it is queued in an in-memory pending-requests map and delivered upon reconnection.

**Grace Period Handling:** When a vendor's socket disconnects (e.g., app backgrounded), the server waits a configurable grace period before marking the vendor offline, preventing false "0 vendors nearby" results during brief disconnections.

## 7. Outcome & Inferences

### 7.1 Outcomes

The prototype successfully demonstrates the core value proposition:

1. **Live vendor discovery** — Buyers can see broadcasting vendors on a real-time map and discover them within a 5 km radius.
2. **Product-level search** — Buyers can search by item name and receive a filtered list of vendors currently selling that product, or search directly by a vendor's Thela ID.
3. **Purchase request flow** — The complete request–accept/reject–location relay cycle works end-to-end. Accepted requests pin the vendor's location on the buyer's map.
4. **Inventory management** — Vendors can add, edit, and delete stock items with quantities; changes auto-sync to the server.
5. **Offline resilience** — Requests and responses are queued when either party is temporarily offline and delivered upon reconnection.
6. **AUJAR abstraction** — The vendor app's GPS layer is abstracted behind the `useAujarBluetooth` hook, providing a clean interface that can be swapped from phone GPS to actual BLE hardware with no changes to the rest of the application.

### 7.2 Inferences

- The WebSocket-based architecture (Socket.io) provides low-latency real-time communication suitable for live GPS tracking and request routing.
- The Haversine-based proximity filter is effective for the current scale; spatial indexing (e.g., MongoDB geospatial queries) would be needed for larger deployments.
- The single-emulator testing scenario exposed the importance of grace-period handling and message queuing — both were added to ensure reliable operation when apps are backgrounded.

- The concept of a dedicated hardware device (AUJAR) is validated at the architecture level: the software abstraction layer is in place and ready for hardware integration.

# 8. Future Works & Development

1. **Automated Calling Service:** Integrate a telephony API (e.g., Twilio) to place automated voice calls to users when a subscribed vendor enters their proximity radius. This ensures notification delivery even on feature phones or when the app is closed.
2. **Server Deployment:** Deploy the bridge server to a cloud platform (e.g., AWS, DigitalOcean) with a production-grade MongoDB Atlas cluster, HTTPS/TLS termination, and environment-based configuration. Migrate in-memory session and queue state to Redis for crash resilience and horizontal scalability.
3. **AUJAR Device Configuration & Packaging:** Complete the hardware prototype by integrating the GPS module and BLE-capable microcontroller onto a custom PCB, programming the firmware for continuous low-power GPS acquisition and BLE transmission, and packaging the electronics into the 3D-printed casing and clamp designs (Figure 1). The final device should mount securely onto a thela, operate for a full 8–12 hour work shift on a single charge, and require no user interaction beyond power-on.
4. **Push Notifications:** Add Firebase Cloud Messaging for in-app alerts when the app is backgrounded.
5. **Vendor Subscription / Follow:** Allow buyers to follow specific vendors and receive proximity-triggered notifications.
6. **Community Relay Mode:** Enable a single phone to relay AUJAR GPS data from multiple nearby vendors simultaneously.
7. **Order History & Analytics:** Persist completed request records and provide vendors with insights on request acceptance rates, peak demand areas, and popular items.

# 9. References


1. Sneha, M., Praveen, K., Myna, K., & Rithika, C. GPS Based Women Security System Using ESP32. *International Journal of Innovative Research in Technology (IJIRT)*, 9(11), 221–224.
2. Nauval, I., Widyanto, R. A., Setiawan, A., Nuryanto, N., Prabowo, N. A., & Purnomo, T. A. (2023). Implementation of GPS for tracking of street vendor. *AIP Conference Proceedings*, 2702(1), 060006.
3. Socket.io Documentation. https://socket.io/docs/v4/
4. Express.js Documentation. https://expressjs.com/
5. MongoDB Mongoose ODM. https://mongoosejs.com/docs/
6. React Native (Expo) Documentation. https://docs.expo.dev/
7. Leaflet.js — Open-source interactive maps. https://leafletjs.com/
8. JSON Web Tokens (JWT) Specification. RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519
9. Haversine Formula for GPS distance calculation. https://en.wikipedia.org/wiki/Haversine_formula
10. React Native BLE PLX — Bluetooth Low Energy library. https://github.com/dotintent/react-native-ble-plx

11. Expo Location API. https://docs.expo.dev/versions/latest/sdk/location/